

Simple SECR workflow

Namkha basic tutorial

Centre for Research into Ecological & Environmental Modelling
University of St Andrews

Session 2 objectives

Main goal: Demonstrate the workflow with a simple example where everything “works”:

1. Create a single-session `capthist` object from camera and detection files
2. Build habitat masks for model fitting and reporting
3. Fit a set of `secr` models
4. Turn fitted models into density, abundance, maps, and checks

The emphasis is the workflow: what each object means and how the scripts connect.

The simple workflow

Each script saves the main object needed by the next step:

01

camera file
detection
file

⇒

capthist

02

survey area
buffer
covariates

⇒

mask

03

(crop for
multisession
model)

⇒

mask (ms)

04

D model
 λ_0/g_0
model

⇒

seccr.fit

05

AICc
abundance
density
maps

Capture histories: script 01_make_capthist.R

01_make_capthist.R creates a capthist object for secr. This contains:

1. the **trapfile**: detector locations and related covariates (a traps object)
2. the **encounter histories**: recorded encounters of animals at detectors (a data frame)

Tasks involved:

- read and clean camera records
- read detection records
- encode effort as trap usage
- attach trap-level covariates
- create and check a capthist
- save the object for later scripts

Capture histories: camera data to traps

Set file paths and CRS used by the workflow

```
my_crs <- "+proj=utm +zone=44 +datum=WGS84 +units=m +no_defs"

traps_file <- "namkha_basic/data/traps.csv"
detections_file <- "namkha_basic/data/detections.csv"

output_file <- "namkha_basic/output/namkha_basic_secr_inputs_capthist.RData"
fig_dir <- "namkha_basic/output/fig"
```

Read in the camera location file (intentionally clean but see later!)

```
cameras <- read.csv(traps_file, stringsAsFactors = FALSE)
```

Capture histories: camera data to traps

	session	trapID	Latitude	Longitude	x	y	UTM.Zone	start_date
1	1	Camera8	30.23653	81.53307	551291	3345115	44R	17/12/2022
2	1	Camera11	30.20199	81.45040	543352	3341253	44R	20/12/2022
3	1	Camera14	30.16731	81.38865	537422	3337389	44R	21/12/2022
4	1	Camera12	30.18575	81.43230	541617	3339447	44R	13/12/2022
5	1	TPC48	30.25107	81.74866	572025	3346844	44R	10/12/2022
6	1	TPC38	29.97843	81.51820	549990	3316508	44R	30/12/2022
7	1	TPC23	29.98457	81.59782	557668	3317226	44R	29/12/2022
8	1	TPC20	30.07531	81.47713	545984	3327226	44R	07/11/2022
	end_date	effort	Topography		Altitude	Water	Note	
1	13/03/2023	86	Cliff		3978	1	Functional	
2	07/02/2023	49	Cliff		4061	0	Functional	
3	16/03/2023	85	Cliff		3959	0	Functional	
4	16/03/2023	93	Cliff		4079	0	Functional	
5	01/06/2023	173	Cliff, Valley floor		4549	1	Functional	
6	14/04/2023	105	Hill slope		4132	1	Functional	
7	12/03/2023	73	Hill slope		3963	0	Functional	
8	18/12/2022	41	Hill slope, Ridgeline		4412	0	Functional	

Capture histories: camera data to traps

Drop malfunctioning or lost cameras

```
cameras <- cameras |> filter(Note == "Functional")
```

Ensure correct date format

```
cameras <- cameras |>  
  mutate(  
    start_date = as.Date(start_date, format = "%d/%m/%Y"),  
    end_date   = as.Date(end_date,   format = "%d/%m/%Y")  
  )
```

Define the overall survey period (here from active cameras)

```
survey_start <- min(cameras$start_date, na.rm = TRUE)  
survey_end   <- max(cameras$end_date,   na.rm = TRUE)
```

Capture histories: camera data to traps

Create the secr traps object

```
traps <- read.traps(  
  data = traps_df |> dplyr::select(trapID, x, y, effort),  
  detector = "count",  
  trapID = "trapID",  
  binary.usage = FALSE  
)
```

Add covariates that might influence detectability of animals

```
covariates(traps)$Topography <- traps_df$Topography  
covariates(traps)$cliff <- traps_df$cliff  
covariates(traps)$hill <- traps_df$hill  
covariates(traps)$valley_stream <- traps_df$valley_stream  
covariates(traps)$Water <- traps_df$Water  
covariates(traps)$Altitude <- traps_df$Altitude
```


Capture histories: detection data to capthist

Read in the detection data (clean; see later for more realistic examples!)

```
capts_raw <- read.csv(detections_file, stringsAsFactors = FALSE)
```

	session	animalID	occasion	date	trapID	sex
1	1	SL1	1	02/16/2023	TPC37	Female
2	1	SL10	1	03/04/2023	TPC14	Female
3	1	SL10	1	05/13/2023	TPC14	Female
4	1	SL11	1	12/18/2022	TPC31	Female
5	1	SL12	1	02/09/2023	TPC25	Male
6	1	SL12	1	11/13/2022	TPC32	Male
7	1	SL13	1	03/04/2023	NRCID296	Female
8	1	SL14	1	03/05/2023	Camera10	Male
9	1	SL14	1	12/10/2022	TPC11	Male
10	1	SL14	1	04/18/2023	TPC11	Male
11	1	SL14	1	01/22/2023	TPC32	Male
12	1	SL15	1	01/12/2023	TPC41	Male

Capture histories: detection data to capthist

Only keep detections in the survey period

```
1 caps_raw <- caps_raw |>
2   mutate(date = as.Date(date, format = "%m/%d/%Y")) |>
3   filter(date >= survey_start, date <= survey_end)
```

Keep variables used by secr

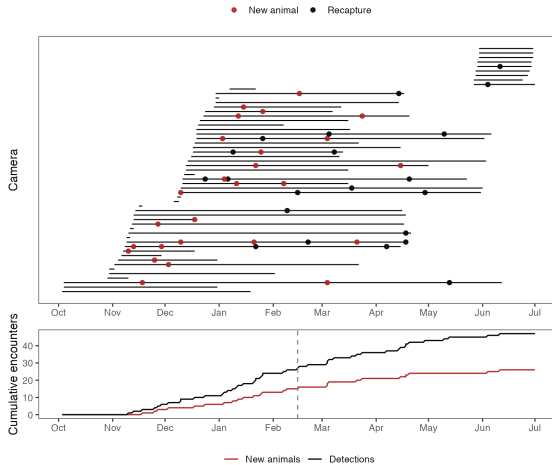
```
caps <- caps_raw |>
  dplyr::select(session, animalID, occasion, trapID, sex)
```

Create and inspect the secr capthist object:

```
1 ch <- make.capthist(captures = caps, traps = traps)
2 verify(ch)
3 summary(ch, terse = TRUE)
```

Capture histories: descriptive checks

Before fitting models, check that effort and detections are coherent:



- do detections occur during active camera periods?
- new individuals distributed over cameras?
- do new individuals keep appearing late in the survey?

Habitat masks: script 02_make_masks.R

02_make_masks.R creates habitat masks (class mask) for fitting and prediction.

Key idea:

- Different masks for model fitting and model prediction
- Model fitting mask needs to include all AC locations animals could be detected from
- Often want to extrapolate to wider area
- *But* where these two masks intersect, useful to share mask points

Implementation:

- Create polygons for (1) close to detectors; (2) extrapolation region
- Find union of these two polygons
- Make mask (points) in this region
- Subset this to form model fitting and prediction masks

Habitat masks: script 02

Tasks involved:

- read the Namkha boundary (extrapolation region)
- build a trap buffer
- create a regular grid of mask points
- add spatial covariates to the mask
- save separate masks for fitting and reporting

Habitat masks: define the region

The full region is the union of the survey boundary and the trap buffer:

```
mask_spacing <- 1500  
mask_buffer  <- 25000
```

Aside: How to choose buffer and spacing?

- Get a rough estimate of σ from capthist
- Rough guesses for buffer = 4σ , spacing = 0.6σ
- Bigger buffers and small spacings are safer but increase run times

```
qnd <- autoini(capthist = ch, mask = make.mask(traps = traps,  
                                                buffer = 75000,  
                                                spacing = 1000))  
  
qnd$sigma * 4    # buffer  
qnd$sigma * 0.6  # spacing
```

Rough buffer: 23704.84

Rough spacing: 3555.726

Habitat masks: define the region

Read in extrapolation region polygon

```
survey_area <- st_read("namkha_basic/data/  
survey_area/Namkha_RM.shp",  
quiet = TRUE) |>  
  st_zm() |>  
  st_transform(my_crs) |>  
  st_geometry()
```

Make traps buffer polygon

```
traps_buffer <- traps_sf %>%  
  st_buffer(mask_buffer) %>%  
  st_union() %>% st_geometry()
```

Full region is the union of the two polygons

```
full_region <- st_union(survey_area, traps_buffer)
```

Habitat masks: create the mask

Build a regular grid

```
mask_grid <- st_make_grid(  
  # add a small buffer so don't lose points on boundary  
  st_buffer(full_region, 10000),  
  cellsize = c(mask_spacing, mask_spacing),  
  what = "centers"  
)
```

Keep points inside the region

```
mask_points <- mask_grid[full_region] |>  
  st_as_sf() |>  
  cbind(st_coordinates(.)) |>  
  st_drop_geometry() |>  
  dplyr::select(x = X, y = Y)
```

Create and inspect the secr mask object

```
mask <- read.mask(data = mask_points, spacing = mask_spacing)
```


Habitat masks: add spatial covariates

Density covariates added to mask because they affect possible AC locations.

Easiest to read in as raster or shape file

```
tri <- rast("namkha_basic/data/spatial_covs/TRI_Namkha.tif")
# remember to project to same CRS as mask
tri <- project(tri, y = my_crs)
names(tri) <- "tri"
mask <- addCovariates(mask, tri)
```

May need some preprocessing (more later on covariates)

```
covariates(mask)$d2hydro <- as.numeric(st_distance(mask_sf, hydro))
```

- mask_model is used for secr.fit()
- mask_survey_area is used for abundance and reporting
- these can but don't have to differ

Model fitting: script 04

04_model_fitting.R fits a set of candidate models.

- start with the null model
- add density covariates such as `std_tri` or `std_d2hydro`
- add detector covariates such as `valley_stream`
- use AIC for model selection

Model fitting: adding covariates

Fitting the null model:

```
m0 <- secr.fit(ch,  
  detectfn = "HHN",  
  mask = mask_model,  
  model = list(D ~ 1, lambda0 ~ 1, sigma ~ 1)  
)
```

m4 has covariates on density and detection

```
m4 <- secr.fit(ch,  
  detectfn = "HHN",  
  mask = mask_model,  
  model = list(D ~ std_tri, lambda0 ~ valley_stream, sigma ~ 1),  
  start = m0  
)
```

Model fitting: adding covariates

Use AICc or AIC to compare the candidate set after fitting

```
AIC(m0, m1, m2, m3, m4, m5, criterion = "AICc")
```

								model
m3								D~1 lambda0~valley_stream sigma~1
m4								D~std_tri lambda0~valley_stream sigma~1
m0								D~1 lambda0~1 sigma~1
m5	D~std_tri + std_d2hydro	lambda0~cliff + valley_stream						sigma~1
m1								D~std_tri lambda0~1 sigma~1
m2								D~std_d2hydro lambda0~1 sigma~1
	detectfn	npar	logLik	AIC	AICc	dAICc	AICcwt	
m3	hazard	halfnormal	4	-106.0014	220.003	221.908	0.000	0.6465
m4	hazard	halfnormal	5	-105.9684	221.937	224.937	3.029	0.1422
m0	hazard	halfnormal	3	-109.1400	224.280	225.371	3.463	0.1145
m5	hazard	halfnormal	7	-103.7172	221.434	227.657	5.749	0.0365
m1	hazard	halfnormal	4	-109.0375	226.075	227.980	6.072	0.0311
m2	hazard	halfnormal	4	-109.0967	226.193	228.098	6.190	0.0293

Interpreting the null model – m_0

Type the name of the model for a model summary

```
secr.fit(capthist = ch, model = list(D ~ 1, lambda0 ~ 1, sigma ~  
  1), mask = mask_model, detectfn = "HHN", verify = FALSE,  
  trace = FALSE)
```

secr 5.4.2, 11:45:50 28 Apr 2026

Detector type	count
Detector number	55
Average spacing	2504.368 m
x-range	534489 576620 m
y-range	3316462 3360989 m

Usage range by occasion

	1
min	2
...	

Interpreting the null model – m_0

Type the name of the model for a model summary

```
...
max 251

N animals      : 26
N detections   : 47
N occasions    : 1
Count model    : Poisson
Mask area      : 667350 ha

Model          : D~1 lambda0~1 sigma~1
Fixed (real)   : none
Detection fn    : hazard halfnormal
Distribution    : poisson
N parameters    : 3
Log likelihood  : -109.14
AIC             : 224.2801
AICc           : 225.371
...
```

Interpreting the null model – m_0

Type the name of the model for a model summary

...

Beta parameters (coefficients)

	beta	SE.beta	lcl	ucl
D	-8.718803	0.2520262	-9.212766	-8.224841
lambda0	-5.965939	0.2929625	-6.540134	-5.391743
sigma	8.711655	0.1437296	8.429950	8.993359

Variance-covariance matrix of beta parameters

	D	lambda0	sigma
D	0.063517229	-0.009673521	-0.01379971
lambda0	-0.009673521	0.085827004	-0.02741592
sigma	-0.013799708	-0.027415917	0.02065819

Fitted (real) parameters evaluated at base levels of covariates

	link	estimate	SE.estimate	lcl	ucl
D	log	1.634827e-04	4.186493e-05	9.975776e-05	2.679149e-04
lambda0	log	2.564636e-03	7.677555e-04	1.444294e-03	4.554030e-03
sigma	log	6.073283e+03	8.774382e+02	4.582270e+03	8.049453e+03

...

Interpreting the null model – m_0

- `coef()` for model coefficients on the link scale

```
coef(m0)
```

	beta	SE.beta	lcl	ucl
D	-8.718803	0.2520262	-9.212766	-8.224841
lambda0	-5.965939	0.2929625	-6.540134	-5.391743
sigma	8.711655	0.1437296	8.429950	8.993359

- `predict()` for model coefficients on the response scale

```
predict(m0)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.634827e-04	4.186493e-05	9.975776e-05	2.679149e-04
lambda0	log	2.564636e-03	7.677555e-04	1.444294e-03	4.554030e-03
sigma	log	6.073283e+03	8.774382e+02	4.582270e+03	8.049453e+03

Interpreting a covariate model – m_4

- `coef()` for model coefficients on the link scale

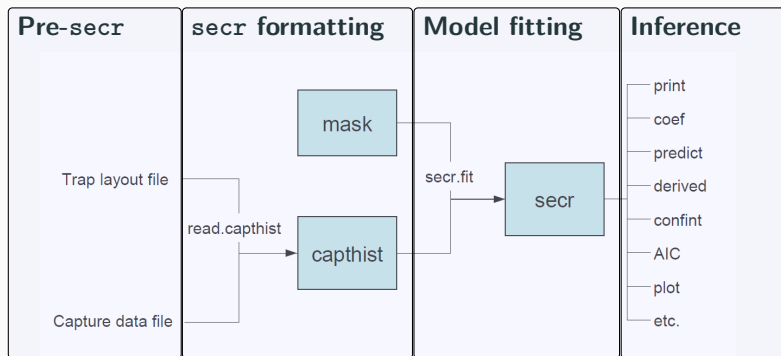
```
coef(m4)
```

	beta	SE.beta	lcl	ucl
D	-8.6926618	0.2604923	-9.203217	-8.18210629
D.std_tri	-0.1021571	0.3950087	-0.876360	0.67204584
lambda0	-5.8202128	0.2963861	-6.401119	-5.23930669
lambda0.valley_stream	-1.3093661	0.6198281	-2.524207	-0.09452529
sigma	8.7232095	0.1455221	8.437991	9.00842768

- `predict()` for model coefficients on the response scale

```
predict(m4)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.678127e-04	4.446608e-05	1.007148e-04	2.796124e-04
lambda0	log	2.966974e-03	8.990400e-04	1.659699e-03	5.303933e-03
sigma	log	6.143866e+03	8.988229e+02	4.619267e+03	8.171663e+03



Post processing functions in secr

A bunch of helper functions for working with model output:

- extract link-scale estimates & associated uncertainty: `coef()`
- extract real-scale estimates & associated uncertainty: `predict()`
- compute the population size for the area: `region.N()`
- plot detection function based on estimates: `plot()`
- plot estimated AC locations: `fxi.contour()`
- many more

Estimation and reporting: script 05

`05_model_results.R` turns fitted models into common outputs.

Tasks involved:

- compare models with AICc
- summarise abundance and density over `mask_survey_area`
- choose the best model for mapping by AICc
- map the survey region and predicted density
- inspect covariate effects on density and detection

Estimation and reporting: build the summary table

Choose models you want to report results for

```
fitted_models <- list(m0 = m0, m1 = m1, m2 = m2,  
                     m3 = m3, m4 = m4, m5 = m5)
```

Choose area you want to report results for:

```
extrap_mask <- mask_survey_area
```

Extracting abundance for all chosen models (extract_abundance is a wrapper function for secr's region.N):

```
abundance_table <- lapply(names(fitted_models),  
                          extract_abundance,  
                          abundance = c("E.N"))
```

Note: abundance = c("E.N") for expected abundance, abundance = c("R.N") for realised abundance

Estimation and reporting: build the summary table

Reshape into nice format:

```
abundance_table <- abundance_table |> bind_rows() |>
  mutate(D = estimate / survey_area_km2,
         Dlcl = lcl / survey_area_km2, Ducl = ucl / survey_area_km2,
         CV = 100 * SE.estimate / estimate)
```

	modelname	estimate	SE.estimate	lcl	ucl	D	Dlcl
1	m0	39.43203	10.097822	24.06157	64.62109	0.01634827	0.009975776
2	m1	38.94529	9.789949	23.97462	63.26422	0.01614647	0.009939726
3	m2	39.30222	9.960437	24.10148	64.09002	0.01629445	0.009992320
4	m3	39.80810	10.261072	24.21485	65.44269	0.01650419	0.010039323
5	m4	40.35744	10.540280	24.39325	66.76941	0.01673194	0.010113290
6	m5	42.00740	11.312175	25.00953	70.55795	0.01741600	0.010368794
		Ducl	CV				
1		0.02679149	25.60817				
2		0.02622895	25.13770				
3		0.02657132	25.34320				
4		0.02713213	25.77634				
5		0.02768218	26.11731				
6		0.02925288	26.92901				

Estimation and reporting: abundance and density

AIC's for chosen models:

```
aic_table <- AIC(m0, m1, m2, m3, m4, m5, criterion = "AICc") |>
  mutate(modelname = row.names(.)) |>
  left_join(abundance_table, by = "modelname")
```

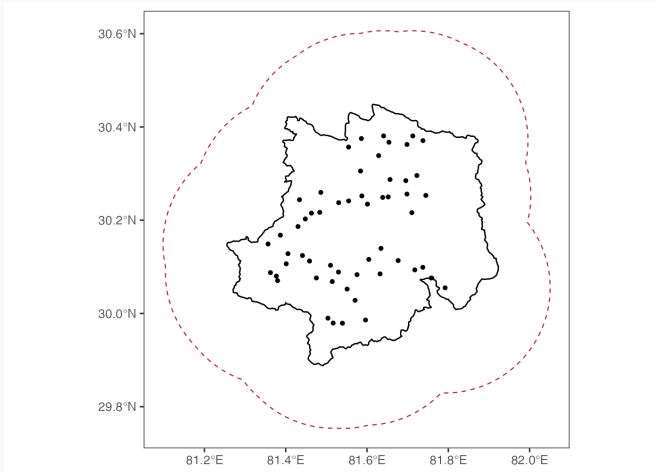
Best model is one with lowest AIC:

```
best_model_name <- aic_table$modelname[which.min(aic_table$AICc)]
best_model <- fitted_models[[best_model_name]]
```

Save results:

```
write.csv(aic_table |>
  select(modelname, model, AIC, AICc, dAICc, AICcwt,
         estimate, lcl, ucl, D, Dlcl, Ducl, CV),
  file = "output/namkha_basic_results_abundance_AIC.csv",
  row.names = FALSE)
```

Estimation and reporting: survey region



Survey boundary, camera locations, and the trap-buffer region used in the workflow.

Estimation and reporting: predicted density

Predicted density at each point in the extrapolation mask:

```
pred_surface <- predictDsurface(best_model,  
                                mask = extrap_mask,  
                                cl.D = FALSE)
```

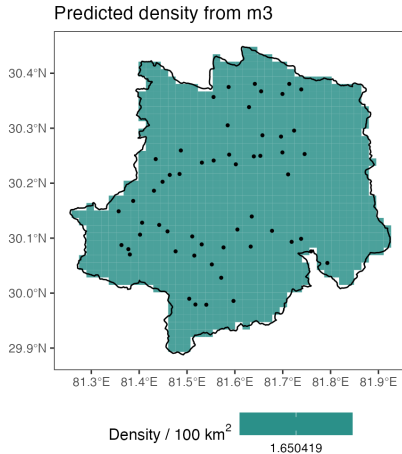
Create grid cells centered on mask points for plotting:

```
density_cells <- st_as_sf(data.frame(extrap_mask),  
                           coords = c("x", "y"), crs = my_crs)  
density_cells <- st_buffer(  
  density_cells,  
  dist = attr(extrap_mask, "spacing") / 2,  
  endCapStyle = "SQUARE",  
  nQuadSegs = 1  
)
```

Add in the densities per 100 km²:

```
density_cells$D_per_100km2 <- covariates(pred_surface)[, "D.0"] * 10000
```

Estimation and reporting: predicted density



Estimation and reporting: density covariate effects

To show how density changes with one focal covariate, choose a model and covariate

```
covplot_model <- m5  
focal_cov <- "std_tri"  
focal_cov_full_name <- "Terrain ruggedness index"
```

Would usually choose the AIC-best model but here using m5 which has two density covariates and two detection covariates

	beta	SE.beta	lcl	ucl
D	-8.4550097	0.4066118	-9.25195416	-7.6580653
D.std_tri	-0.2652649	0.3984637	-1.04623940	0.5157097
D.std_d2hydro	0.4465780	0.7186844	-0.96201753	1.8551736
lambda0	-6.0508404	0.3335679	-6.70462142	-5.3970594
lambda0.cliff	0.6394071	0.3173501	0.01741238	1.2614018
lambda0.valley_stream	-1.3472480	0.6278909	-2.57789158	-0.1166045
sigma	8.6865962	0.1448344	8.40272591	8.9704664

Estimation and reporting: density covariate effects

Create a range of potential values for the focal covariate:

```
newdata <- data.frame(  
  focal_values = seq(  
    min(covariates(extrap_mask)[, focal_cov], na.rm = TRUE),  
    max(covariates(extrap_mask)[, focal_cov], na.rm = TRUE),  
    length.out = min(500, nrow(extrap_mask))  
  )  
)  
names(newdata)[1] <- focal_cov
```

Other covariates are set to their mean or mode (not shown here)

Estimation and reporting: density covariate effects

Predicting densities at new covariate values is a bit fiddly.

- Use a temporary mask so `predictDsurface` can evaluate the density model.

```
pred_mask <- subset(extrap_mask, subset = 1:nrow(newdata))
```

- Replace covariate values with the ones we want

```
for (covname in names(newdata)) {  
  covariates(pred_mask)[, covname] <- newdata[[covname]]  
}
```

- Now predict density

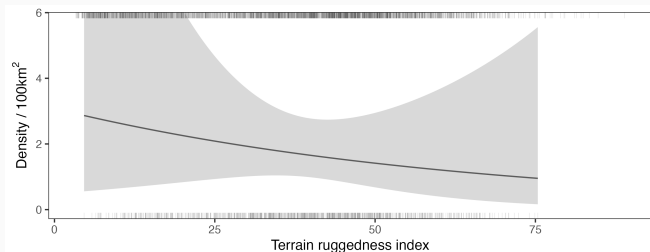
```
tmp <- predictDsurface(covplot_model, mask = pred_mask, cl.D = TRUE)
```

Estimation and reporting: density covariate effects

- Extract observed covariate values on mask and extrapolation region to assess covariate coverage and include this on plot

```
x_surveyed <- data.frame(xt = c(covariates(mask_model)[, plot_cov]))  
x_full <- data.frame(xt = c(covariates(extrap_mask)[, plot_cov]))
```

- Finally plot:



Estimation and reporting: encounter rate covariate effects

Identify which covariates appear in the encounter-rate part of the chosen model

```
model_covs <- covplot_model$betanames
lam0_covs <- sub("^lambda0\\.\"", "",
                 model_covs[grepl("^lambda0\\.\"", model_covs)])
lam0_covs
```

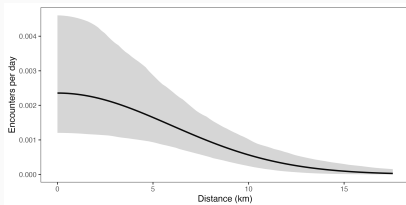
```
[1] "cliff"          "valley_stream"
```

Set possible values for any λ_0 covariates **in same order as** lam0_covs

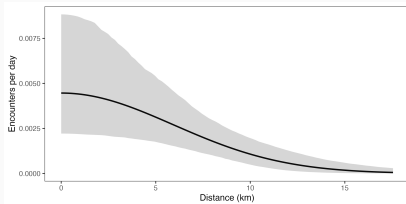
```
lam0_covs_vals <- c(1, 0) # for cliff
lam0_covs_vals <- c(0, 1) # for valley/stream
lam0_covs_vals <- c(0, 0) # for baseline (other)
```

Estimation and reporting: encounter rate covariate effects

Plot for baseline:



Plot for cliff:



Workflow recap

1. Read in camera and detection csv's and create `capthist`
2. Build masks for model fitting and reporting (keep these concepts separate)
3. Fit candidate `secr` models
4. Produce interpretable output: density, abundance, maps, covariate effects

What did we miss?

1. Survey period too long for population closure
2. Barriers to movement
3. Other covariates
4. Sex-dependent parameters
5. Perfectly clean data doesn't exist
6. Probably lots more!

Some of these are covered in the next sections